Sequential Learning with Deep Neural Networks

RNN and LSTM Applied to Yelp Sentiment Classification

Machine Learning and Neural Networks — Individual Assignment

Bhoovika Venkatachalam | 9972CZHG

Abstract

This report examines how architectural choices in deep learning affect performance on sequential text tasks. Beginning with a baseline Deep Neural Network (DNN) operating on Bag-of-Words (BoW) input, it traces the need for Recurrent Neural Networks (RNN) and ultimately Long Short-Term Memory (LSTM) networks, which address the core limitation of order-blindness inherent in standard DNNs. Both architectures are benchmarked on the Yelp Review Polarity dataset. The LSTM achieved 93.62% test accuracy compared to 88.21% for the DNN, with the gap attributable primarily to architectural alignment rather than raw model size.

1. Introduction

Modern deep learning has achieved strong results across computer vision, speech recognition, and natural language processing. Despite this breadth, no single architecture excels uniformly — performance depends heavily on how well the model's design reflects the structure of the input data.

Earlier work in this series covered the Single Layer Perceptron and Multilayer Perceptron. Both assume that input features are unordered and independent, an assumption that holds for tabular data but fails for text, where meaning depends on word order. This report picks up at that boundary, introducing RNN and LSTM as architectures purpose-built for sequential input, using Yelp sentiment classification as the practical testbed.

2. Dataset Description

The Yelp Review Polarity dataset (Zhang et al., 2015), available via Hugging Face, contains 598,000 consumer restaurant reviews labelled as binary sentiment: negative (1–2 stars) and positive (4–5 stars). Three-star neutral reviews are excluded to keep the sentiment signal unambiguous.

The dataset is evenly split — 280,000 samples per class in the 560,000-sample training set, and 38,000 samples in the test set. This balance means accuracy is a meaningful and unbiased metric. Review lengths range from a handful of words to several hundred tokens, which directly motivates the padding and truncation strategy discussed next.

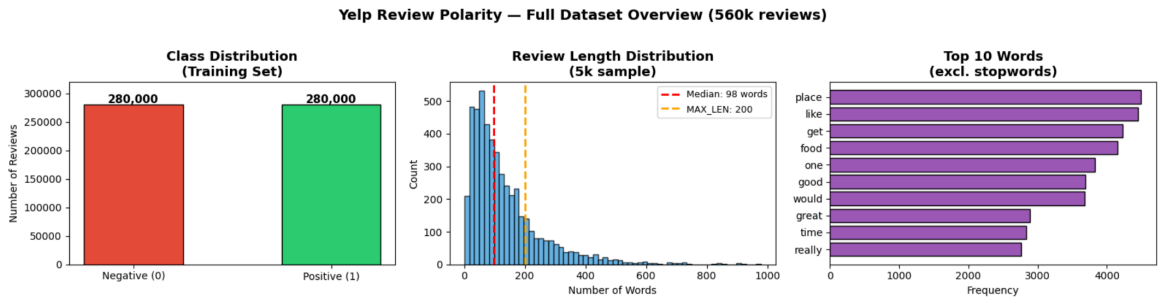

Figure 1: Yelp Review Polarity — Class distribution, review length distribution), and top 10 words.

3. Preprocessing Pipeline

Text Cleaning

Each review is lowercased, stripped of punctuation via regex, tokenised into individual words, and filtered to remove high-frequency stopwords (e.g. 'the', 'a', 'is') that carry no sentiment signal. Critically, negation words like 'not' are retained — 'food not good' carries different meaning than 'food good', and removing negations would corrupt the label.

Tokenisation and Vocabulary

Words are mapped to integer indices drawn from a 30,000-word vocabulary fitted on the training set only. Words outside this vocabulary receive an out-of-vocabulary token. Fitting on training data exclusively prevents test set information from leaking into the model.

Sequence Formatting and the BoW vs Sequence Split

All sequences are padded or truncated to 200 tokens. This length was chosen based on the review length distribution — it captures the bulk of sentiment content without generating excessive padding overhead. The 200-token median aligns with roughly 98 words, confirming the length is sufficient.

A critical design distinction governs how each model consumes text. The DNN receives a Bag-of-Words vector: a fixed-width count of word occurrences with no positional information. The LSTM receives a padded integer sequence preserving word order. The phrase 'food was not good' and 'food was good, not bad' produce nearly identical BoW vectors despite opposite sentiments — a distinction only the LSTM can resolve.

4. Model Architecture: DNN, RNN, and LSTM

4.1 Deep Neural Network

A DNN extends the MLP by stacking multiple hidden layers to extract progressively abstract features from input. Shallower layers capture low-level patterns; deeper layers combine them into higher-level representations.

The DNN in this study uses ten fully connected hidden layers with widths 512, 512, 256, 256, 128, 128, 64, 64, 32, and 16 neurons. Each hidden layer is followed by Batch Normalisation, which stabilises activation distributions and speeds convergence, and Dropout regularisation at rate 0.4, which prevents co-adaptation of neurons. The input is an 8,000-feature BoW vector; the output is a single sigmoid unit for binary classification.

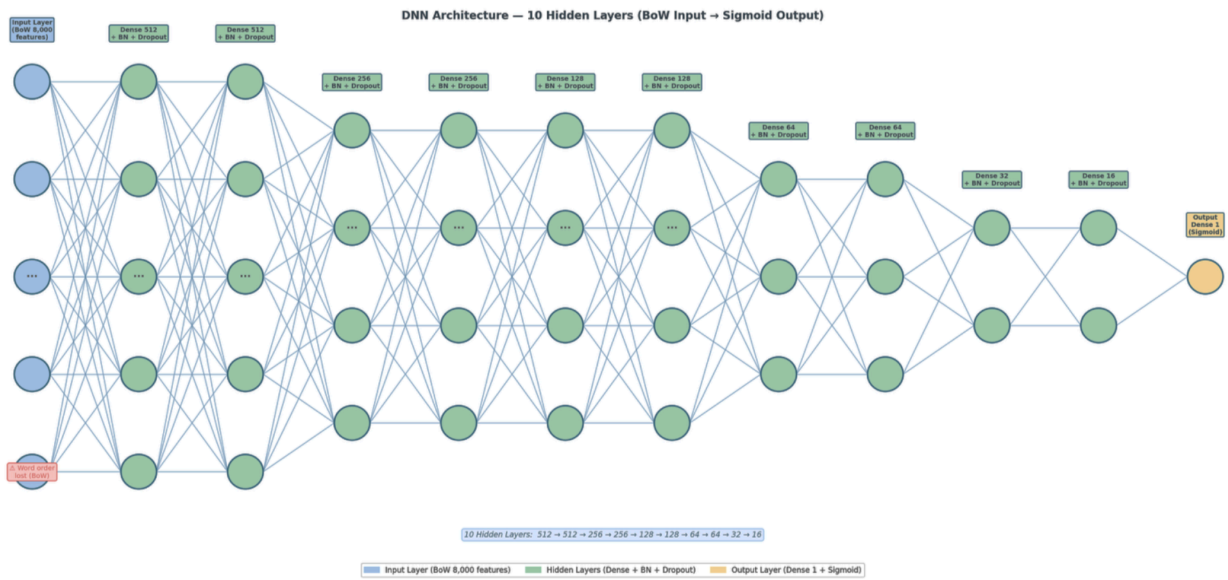

Figure 2: DNN Architecture

Despite this depth, the DNN has a structural blind spot: because BoW discards word order before the network even begins processing, no amount of additional layers can recover it. The limitation is in the input representation, not the model depth.

4.2 Recurrent Neural Network

RNNs were introduced to process variable-length sequences directly. Rather than consuming the full input as a flat vector, an RNN reads one token at a time while maintaining a hidden state that carries accumulated context forward. At each step t, the hidden state h_t is computed from both the current input token x_t and the prior hidden state h_{t-1}, allowing the model to build a running contextual summary of the sequence.

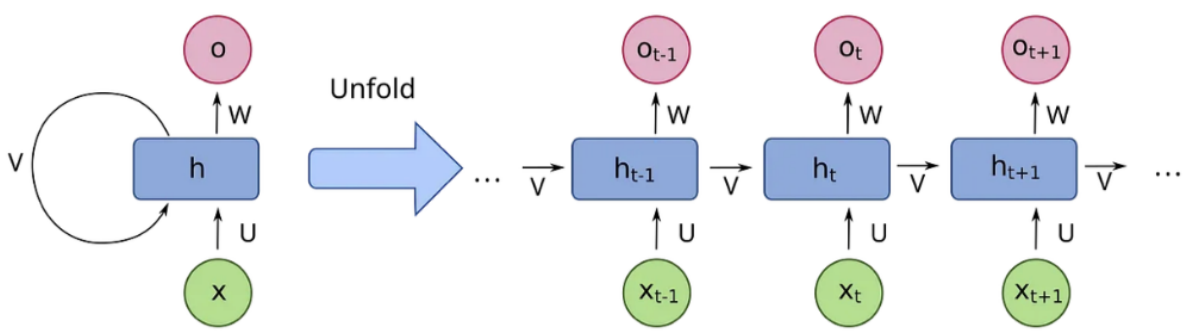

Figure 3: RNN Architecture

However, vanilla RNNs suffer from vanishing gradients during backpropagation through time. As gradients are propagated backward across many timesteps, they shrink exponentially, causing the model to lose sensitivity to early tokens. In long reviews, a negation near the start may become invisible to the model by the time it processes sentiment words near the end.

4.3 Long Short-Term Memory

LSTM networks (Hochreiter & Schmidhuber, 1997) were designed to overcome this vanishing gradient limitation. The core innovation is a dedicated cell state — a separate long-range memory channel that can carry information across the full sequence without exponential decay. Three learned gates govern access to this cell state:

The forget gate decides what fraction of the existing cell state to erase. The input gate determines what new information to write in. The output gate controls what portion of the updated cell state is exposed as the current hidden state. Each gate has its own learnable weight matrices and is trained end-to-end, so the model learns memory management directly from data rather than through hand-coded rules.

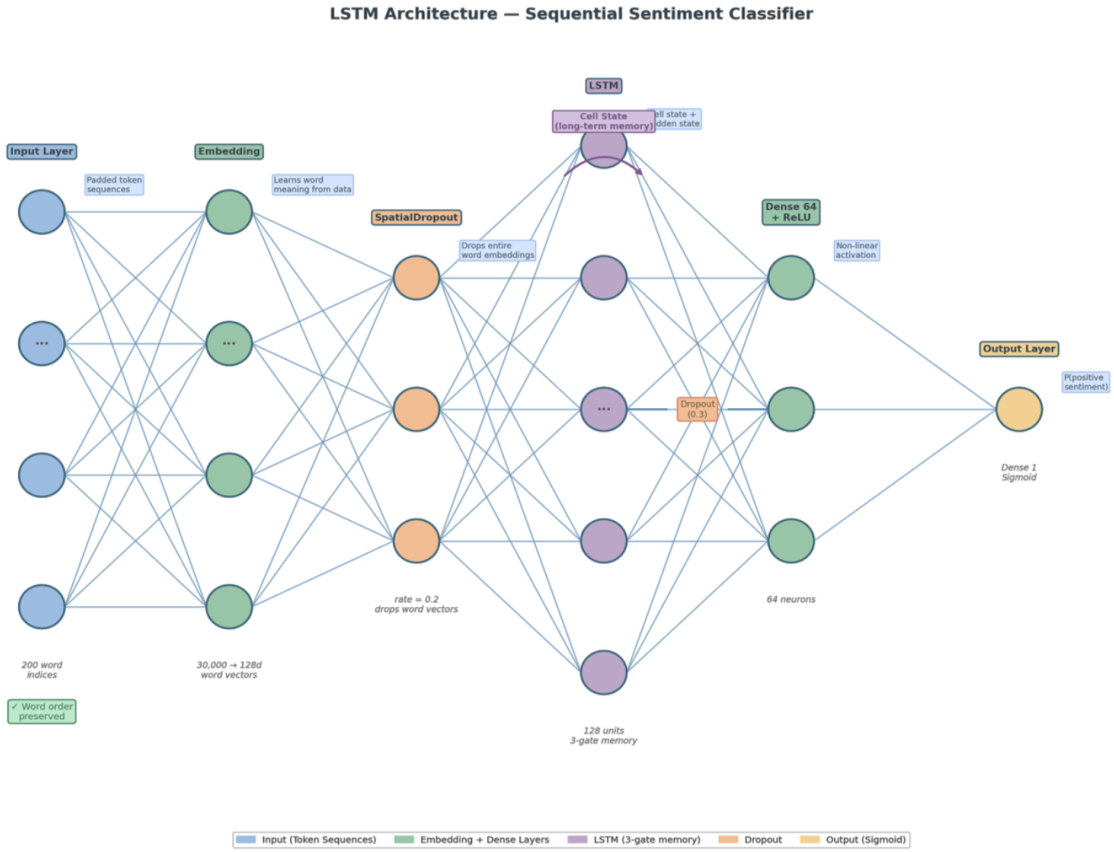

Figure 4: LSTM Architecture

In this study, the LSTM architecture is: Input (padded sequences) → Embedding (30,000 vocab × 128 dims) → SpatialDropout(0.2) → LSTM(128 units) → Dense(64, ReLU) → Dropout(0.5) → Dense(1, Sigmoid). SpatialDropout drops entire word vectors rather than individual values, which is more appropriate for sequential embeddings than standard dropout.

4.4 Architectural Comparison

Feature	DNN	LSTM
Input format	Bag-of-Words vector	Padded integer sequences
Word order	Discarded	Fully preserved
Memory mechanism	None — stateless	Cell state + gated updates
Long-range dependency	Not possible	Handled via gating
Core strength	Structured, orderless data	Sequential, context-sensitive text
Core weakness	Blind to word order and negation	Slower to train, higher compute cost

Table 1: DNN vs LSTM architectural comparison

5. Training Configuration

Due to memory constraints, the DNN was trained on a stratified 50,000-sample subset (25,000 per class). Training the DNN on the full 560,000-sample set caused memory overflow and visible overfitting. The LSTM, with its smaller parameter footprint relative to input size, trained on the complete 560,000 samples. This asymmetry is a genuine experimental constraint and is accounted for in the result interpretation.

Hyperparameter	Value	Rationale
Vocabulary size	30,000	Covers common and meaningful rare words
Max sequence length	200 tokens	Captures majority of sentiment content
Embedding dimensions	128	Sufficient resolution, avoids overfitting
LSTM units	128	Balances memory capacity and training speed
Batch size	256	Stable gradient estimates; efficient on GPU
Max epochs	7	Convergence reached without overfitting
DNN dropout rate	0.4	Aggressive regularisation for small subset
DNN BoW features	8,000	Core vocabulary within memory budget

Table 2: Hyperparameter settings and justifications

The DNN applies Batch Normalisation and Dropout(0.4) after each hidden layer. The LSTM uses SpatialDropout on the embedding layer and two training callbacks: EarlyStopping (patience = 3, restores best weights) and ReduceLROnPlateau (halves learning rate on validation loss plateau), together preventing overfitting without manual epoch tuning.

6. Results and Discussion

Model	Training Samples	Test Accuracy
DNN (10 layers, BoW)	50,000 (subset)	88.21%
LSTM (Sequential)	560,000 (full)	93.62%

Table 3: Model performance on the 38,000-sample held-out test set

The LSTM outperforms the DNN by 5.41 percentage points. However, given the asymmetric training conditions, the accuracy gap cannot be attributed to architecture alone. The DNN's 88.21% result was only achievable after reducing to the 50,000-sample subset — on the full dataset it overfit severely despite 560,000 training samples, which itself reveals the architectural mismatch: the problem was not data scarcity but BoW's inability to capture sequential structure.

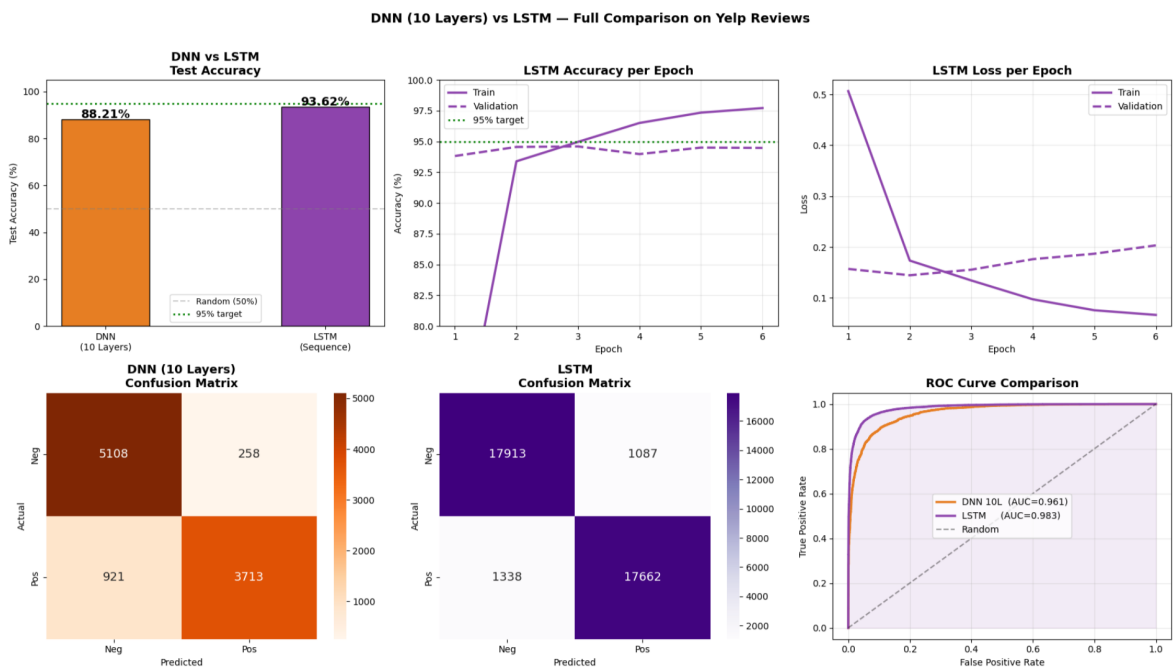

Figure 5: DNN vs LSTM comparison, LSTM training curves, confusion matrices, and ROC curve.

The LSTM's training and validation curves converged without divergence, indicating genuine generalisation rather than memorisation. A live demonstration with the review 'The waiter was rude but the pasta was honestly the best I have ever had' showed the LSTM correctly classifying positive sentiment by recognising the contrastive role of 'but', while the DNN, working from word counts alone, struggled with this mixed polarity input.

The ROC curves support the same conclusion: LSTM AUC = 0.983 vs DNN AUC = 0.961. Both models demonstrate strong discriminative ability, but the LSTM consistently holds the advantage on this sequential task.

7. Limitations and Future Work

The primary limitation is the non-equivalent training setup: comparing a model trained on 50,000 samples to one trained on 560,000 conflates architectural and data-scale effects. A controlled evaluation on identical subsets would isolate the architectural contribution.

The LSTM also struggles with sarcasm and irony — surface-positive language masking negative intent (e.g. 'Oh great, another cold meal') is difficult to resolve from sequential context alone. Reviews exceeding 200 tokens are truncated, potentially losing sentiment-relevant content.

Future directions include Bidirectional LSTM, which reads sequences in both directions for richer context; attention mechanisms, which allow the model to weight relevant timesteps rather than compressing all information into a single vector; and Transformer architectures such as BERT, which apply self-attention across all tokens simultaneously and currently represent the state of the art for text classification tasks.

8. Conclusion

This study demonstrates that architectural alignment with task structure matters more than raw model complexity. A 10-layer DNN with millions of parameters cannot overcome the fundamental constraint imposed by BoW input: word order is gone before any learning begins. The LSTM resolves this at the design level via gated sequential processing, yielding a 5.41-point accuracy advantage on a task where word order is essential to meaning.

The core takeaway is that choosing a model requires reasoning about what information the task demands and whether the chosen architecture can represent it — not simply scaling up existing designs.

References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. https://www.deeplearningbook.org/
2. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
3. Ioffe, S., & Szegedy, C. (2015). Batch normalisation: Accelerating deep network training. ICML, 448–456.
4. Olah, C. (2015). Understanding LSTM networks. Colah's Blog. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
5. Srivastava, N., Hinton, G., et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. JMLR, 15(1), 1929–1958.
6. Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. NeurIPS, 28, 649–657.